MONICAAJHOK10362028

7.24

```
CREATE DATABASE Projects;
Query OK, 1 row affected (0.007 sec)

MariaDB [(none)]> use projects;
Database changed

MariaDB [projects]>  CREATE TABLE Employee(
empNo VARCHAR (5) PRIMARY KEY ,
 fName VARCHAR (20),
lName VARCHAR (20),
 address VARCHAR (100),
 DOB DATE ,
 sex CHAR(1),
 position VARCHAR (30),
 deotNo  VARCHAR (5),

 CHECK (sex IN ("M", "F")),

CHECK (position IN
 ("Manager",
 "Team Leader",
"Analyst",

 "Software Developer"))
);
Query OK, 0 rows affected (0.012 sec)

MariaDB [projects]>  CREATE TABLE Department (
deptNo VARCHAR (5) PRIMARY KEY,
deptName VARCHAR (50),
mgrEmpNo VARCHAR (5),

FOREIGN KEY (mgrEmpNO)
REFERENCES Employee(empNO)
 );
Query OK, 0 rows affected (0.012 sec)

MariaDB [projects]> CREATE TABLE Project (
projNo VARCHAR (10) PRIMARY KEY ,
projName VARCHAR (50),
deptNo VARCHAR (5)
);
Query OK, 0 rows affected (0.011 sec)
```

```
MariaDB [projects]>  CREATE TABLE Workson  (
 empNo VARCHAR (5),
 projNo VARCHAR (10),
 dateWorked DATE ,
hoursWorked INTEGER,

 PRIMARY KEY (empNo, projNo, dateWorked),

CHECK (hoursWorked BETWEEN 0 AND 40)
 );
Query OK, 0 rows affected (0.012 sec)


 ALTER TABLE Employee
    -> CHANGE deotNo deptNo VARCHAR(5);
Query OK, 0 rows affected (0.015 sec)
Records: 0  Duplicates: 0  Warnings: 0

DESCRIBE Employee;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| empNo    | varchar(5)   | NO   | PRI | NULL    |       |
| fName    | varchar(20)  | YES  |     | NULL    |       |
| lName    | varchar(20)  | YES  |     | NULL    |       |
| address  | varchar(100) | YES  |     | NULL    |       |
| DOB      | date         | YES  |     | NULL    |       |
| sex      | char(1)      | YES  |     | NULL    |       |
| position | varchar(30)  | YES  |     | NULL    |       |
| deptNo   | varchar(5)   | YES  |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
8 rows in set (0.040 sec)

 ALTER TABLE Project ADD CONSTRAINT unique_projName UNIQUE (projName);
Query OK, 0 rows affected (0.016 sec)
Records: 0  Duplicates: 0  Warnings: 0


7.25 Create a View of Projects Managed by Female Managers



MariaDB [projects]> CREATE VIEW FemaleManagerProjects AS
    -> SELECT p.projNo,
    ->        p.projName,
    ->        p.deptNo
    -> FROM Project p,
    ->      Department d,
    ->      Employee e
```

```
    -> WHERE p.deptNo = d.deptNo
    -> AND d.mgrEmpNo = e.empNo
    -> AND e.sex = 'F'
    -> ORDER BY p.projNo;
Query OK, 0 rows affected (0.011 sec)
```

7.26 Create a View with empNo, fName, lName, projName and hoursWorked

```
MariaDB [projects]> CREATE VIEW EmployeeProjectHours AS
    -> SELECT e.empNo,
    ->        e.fName,
    ->        e.lName,
    ->        p.projName,
    ->        w.hoursWorked
    -> FROM Employee e,
    ->      Project p,
    ->      WorksOn w
    -> WHERE e.empNo = w.empNo
    -> AND p.projNo = w.projNo;
Query OK, 0 rows affected (0.009 sec)
```

7.27(a)

Given View

```
MariaDB [projects]> CREATE VIEW EmpProject(empNo, projNo, totalHours)
    -> AS
    -> SELECT w.empNo,
    ->        w.projNo,
    ->        SUM(hoursWorked)
    -> FROM Employee e,
    ->      Project p,
    ->      WorksOn w
    -> WHERE e.empNo = w.empNo
    -> AND p.projNo = w.projNo
    -> GROUP BY w.empNo, w.projNo;
Query OK, 0 rows affected (0.009 sec)

MariaDB [projects]> SELECT *
    -> FROM EmpProject;
Empty set (0.005 sec)
```

7.27(b)

```
MariaDB [projects]>  SELECT projNo
```

```
    -> FROM EmpProject
    -> WHERE projNo = 'SCCS';
Empty set (0.002 sec)
```

7.27(c)

```
MariaDB [projects]> SELECT COUNT(projNo)
    -> FROM EmpProject
    -> WHERE empNo = 'E1';
+---------------+
| COUNT(projNo) |
+---------------+
|             0 |
+---------------+
1 row in set (0.001 sec)
```

7.27(d)

The textbook query is:

```
MariaDB [projects]>SELECT empNo,
    ->        SUM(totalHours)
    -> FROM EmpProject
    -> GROUP BY empNo;
Empty set (0.001 sec)
```

7.28 Materialized View Question

```
MariaDB [projects]> CREATE TABLE Part
    -> (
    ->     partNo VARCHAR(10),
    ->     contract VARCHAR(10),
    ->     partCost DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.011 sec)

MariaDB [projects]> CREATE VIEW ExpensiveParts(partNo)
    -> AS
    -> SELECT DISTINCT partNo
    -> FROM Part
    -> WHERE partCost > 1000;
Query OK, 0 rows affected (0.009 sec)
```

Case 1: Insert

```
INSERT INTO Part
    -> VALUES ('P4','C1',1500);
Query OK, 1 row affected (0.003 sec)
```

Case 2: Delete

```
MariaDB [projects]> DELETE FROM Part
    -> WHERE partNo='P3';
Query OK, 0 rows affected (0.002 sec)
```

Case 3: Update

```
MariaDB [projects]> UPDATE Part
    -> SET partCost=1300
    -> WHERE partNo='P2';
Query OK, 0 rows affected (0.001 sec)
Rows matched: 0  Changed: 0  Warnings: 0


MariaDB [projects]> INSERT INTO Part
    -> VALUES ('P5','C1',2000);
Query OK, 1 row affected (0.002 sec)

MariaDB [projects]> UPDATE Part
    -> SET partCost=1500
    -> WHERE partNo='P6';
Query OK, 0 rows affected (0.001 sec)
Rows matched: 0  Changed: 0  Warnings: 0

MariaDB [projects]> DELETE FROM Part
    -> WHERE partNo='P1'
    -> AND contract='C1';
Query OK, 0 rows affected (0.001 sec)
```

A materialized view is similar to a table that contains a snapshot of the results of a query.

How to maintain it:

The materialized view should change if the base table Part changes (inserts, updates, deletes).

Suppose a new part is inserted such that its cost is greater than 1000, it is supposed to be displayed on the view. When deleting a part, it should be removed from the view. If the cost of a part is changed, the view should update to reflect the new cost.

Not using the base table:

If the database system has a log of all changes (such as inserts, updates, and

deletes), then the materialized view can be updated directly from the log.

This will save you from having to rescan the entire Part table again – you only apply the changes.

Example: If a part’s cost changes from 900 to 1200, the system can add that part to the view immediately using the log, without checking the entire table.

When possible:

With incremental refresh (only apply changes, no rebuild), you can maintain the view without accessing the base table.

This is efficient because it saves time and there is unnecessary re-reading of all data.